

AI-Based Inertial Navigation for GPS-Denied Environments

A Learning-Augmented Inertial and Visual-Inertial Approach to Resilient UAV State Estimation

Technical Thesis • Defence Application: Navigation under GPS Jamming, Denial, and Spoofing

Abstract

Satellite positioning is the single point of failure in autonomous flight: when GPS is jammed, denied, or spoofed — a routine condition in contested airspace — a UAV must estimate its own heading and motion from onboard sensors alone, and any drift in that estimate flies the aircraft off course. This thesis develops and validates that onboard capability on public, ground-truthed flight data (EuRoC MAV; TUM-VI for cross-sensor tests). The core contribution, **Approach B**, is a lightweight (~19k-parameter) neural gyroscope denoiser fused into a classical Error-State Kalman Filter: on held-out sequences it removes the systematic gyro bias the filter is structurally blind to, cutting open-loop attitude drift by ~98% and recovering near-oracle fused accuracy from data alone, with the win shown stable across random seeds and across environments. A cross-sensor study establishes the honest limit — the learned correction is sensor-specific — and demonstrates that a short per-IMU fine-tune closes the gap. **Approach A** adds a camera (OpenVINS stereo VIO, ~6 cm) to bound absolute position, and reports two honest negative results on a naive learned visual front-end. Finally, a GPS-denied flight demonstrator and a drift-vs-denial-time characterization quantify the operational payoff: after 60 s of GPS denial the learned system holds roughly three times tighter than a conventional inertial baseline. The remaining step to a fielded system is an on-board (edge/realtime) deployment, identified as future work.

Reproducible prototype — all figures and numbers regenerated from in-repo experiment outputs.

Compiled 14 July 2026.

Contents

Contents	2
Chapter 1 — Introduction	4
1.1 Motivation	4
1.2 Thesis statement	4
1.3 Contributions	4
1.4 Scope and methodology	4
Chapter 2 — Background and Related Work	6
2.1 Inertial navigation and observability	6
2.2 The Error-State Kalman Filter	6
2.3 Learned inertial methods	6
2.4 Visual-inertial odometry	6
2.5 Datasets	6
Chapter 3 — System Overview and Method	7
3.1 Two-stage architecture: learn, then filter	7
3.2 Training signal (no clean-gyro label exists)	7
3.3 Evaluation protocol	7
3.4 Implementation	7
Chapter 4 — Approach B — Learned Inertial Navigation	9
4.1 Phase 0 — Environment and data pipeline	9
4.2 Phase 1 — Naive integration (the motivation)	9
4.3 Phase 2 — Classical ESKF baseline (no AI)	9
4.4 Phase 3 — Gyro-denoising CNN (open-loop)	10
4.5 Phase 4 — Fusion: denoiser + ESKF	11
Chapter 5 — Robustness and Hardening	13
5.1 Multi-seed stability	13
5.2 Cross-environment generalization	13
5.3 Cross-sensor transfer — the honest limit and its fix	13
Chapter 6 — Approach A — Visual-Inertial Navigation	15
6.1 A0 — Camera pipeline	15
6.2 A1 — OpenVINS stereo-VIO baseline	15

6.3 A2 — Learned visual front-end (two honest negatives)	15
Chapter 7 — GPS-Denied Demonstrator and Drift Characterization	17
7.1 The flight demonstrator	17
7.2 D3 — Drift versus denial-time	17
Chapter 8 — Deployment Considerations	19
8.1 D1 — Edge / realtime (remaining, needs hardware)	19
8.2 D2 — Real-IMU transfer (addressed, Chapter 5)	19
8.3 D3 — Drift under sustained denial (addressed, Chapter 7)	19
8.4 D4 — No-magnetometer assumption (addressed)	19
8.5 Engineering note — training-loop vectorization	19
Chapter 9 — Discussion, Limitations, and Future Work	20
9.1 Why the method works	20
9.2 Limitations	20
9.3 Future work	20
Chapter 10 — Conclusion	21
References	22

Chapter 1 — Introduction

1.1 Motivation

An autonomous UAV normally knows where it is because a GNSS (GPS) receiver tells it. In contested, electronic-warfare, or denied environments that signal cannot be trusted: it may be *jammed* (drowned out), *denied* (absent), or *spoofed* (falsified). Jamming and spoofing of drones are live, named problems in modern conflict, contested logistics, and counter-UAS operations. When the fix disappears, the aircraft must fall back on sensors an adversary cannot easily corrupt and continue to estimate its own state — heading, velocity, and position — well enough to keep flying its mission or return safely.

The always-available sensor is the Inertial Measurement Unit (IMU): a three-axis gyroscope and three-axis accelerometer. But IMU signals are corrupted by slowly-varying *bias* and random *noise*, and naive integration of them makes the estimated orientation and position diverge within seconds. The most operationally dangerous component is **heading (yaw)**: it is physically *unobservable* to an IMU-only filter (gravity pins roll and pitch but carries no heading information), so any gyro yaw-bias integrates into unbounded heading error — and on a vehicle a wrong heading steers the entire trajectory off course.

1.2 Thesis statement

A small neural network can learn the systematic error of a specific IMU and subtract it — including the unobservable yaw bias — before a classical filter runs, giving state-estimation accuracy the filter cannot achieve on its own, and doing so cheaply enough to run onboard. A camera, when available, then bounds absolute position that inertial sensing alone cannot.

1.3 Contributions

This thesis makes the following contributions, each validated on public data and reproduced in the accompanying repository:

- **A learned-inertial estimator (Approach B).** A ~19k-parameter gyro denoiser fused into a 15-state ESKF that cuts open-loop attitude drift by ~98% and recovers near-oracle fused accuracy on held-out sequences (Chapters 3–4).
- **A rigorous robustness study.** Multi-seed stability, cross-environment generalization, and a cross-sensor transfer analysis that states the method's honest limit and demonstrates the per-IMU fine-tune that fixes it (Chapter 5).
- **A visual-inertial position layer (Approach A).** An OpenVINS stereo-VIO baseline (~6 cm), and two honest negative results on a naive learned visual front-end that scope where learned VIO does and does not help (Chapter 6).
- **An operational characterization.** A GPS-denied flight demonstrator and a drift-vs-denial-time curve quantifying the payoff (~3× tighter at 60 s of denial), plus a deployment analysis (Chapters 7–8).

1.4 Scope and methodology

The work uses **public datasets only** (no custom flight hardware at this stage); the deliverable is a correct, well-instrumented prototype. A guiding methodological rule is applied throughout: *establish a working classical baseline first, then insert the neural component*, so the exact contribution of the learning is always measurable and the filter and network are never debugged simultaneously. All comparisons are made on the realistic same-information setting (no oracle knowledge handed to the learned side), and negative results are reported at full strength.

Operational assumption — no magnetometer. A magnetometer would normally observe heading, but it is easily jammed and unreliable near motors and airframe currents, so it is treated as unavailable. Heading is held by the gyro alone — which is precisely why removing the gyro yaw bias is the central result (Chapter 8, D4).

Chapter 2 — Background and Related Work

2.1 Inertial navigation and observability

Strapdown inertial navigation integrates gyroscope angular rates to track orientation and doubly-integrates accelerometer specific force (minus gravity) to track velocity and position. The Achilles' heel is error growth: a constant gyro bias produces an orientation error that grows linearly, which mis-rotates the gravity subtraction and injects an acceleration error that grows the position error cubically in time. An aided filter constrains part of this — an accelerometer gives an absolute gravity (tilt) reference that observes roll and pitch — but rotation about the gravity vector (heading/yaw) remains unobservable to an IMU+gravity system. This observability structure is the technical crux of the entire thesis.

2.2 The Error-State Kalman Filter

The Error-State (indirect) Kalman Filter tracks a nominal state and a small *error* state, keeping the linearization valid and the orientation error minimal (three parameters rather than a constrained quaternion). This work follows the local-error convention of Sòla's quaternion-kinematics formulation [4], with a 15-dimensional error state (attitude, velocity, position, gyro bias, accelerometer bias).

2.3 Learned inertial methods

Deep networks have been used to regress the systematic component of IMU error. The design here follows Brossard *et al.* [1], who denoise gyroscope signals with a dilated convolutional network supervised by integrated orientation increments; related lines include AI-IMU dead-reckoning and the RONIN / TLIO / RIANN family of learned-inertial odometry [5]. The distinctive design choices adopted — a bounded, zero-initialised correction so training starts exactly at the raw gyro — are detailed in Chapter 3.

2.4 Visual-inertial odometry

When a camera is available, visual-inertial odometry (VIO) makes absolute scale and position observable. Approach A uses OpenVINS [2], an open-source filter-based (MSCKF) stereo VIO, as a strong classical baseline, and investigates replacing its hand-engineered feature front-end with a learned detector/matcher (SuperPoint [6] + LightGlue [7]). The back-end estimator is left unchanged so that any difference is attributable to the front-end.

2.5 Datasets

EuRoC MAV [3] provides 200 Hz IMU (Analog Devices ADIS16448), stereo imagery, and millimetre-accurate Vicon/Leica ground truth including the true gyro/accel biases, across two Vicon rooms and a machine hall at easy/medium/difficult motion levels. **TUM-VI** [8] is used for the cross-sensor study; its room sequences use a Bosch BMI160-class consumer MEMS IMU — a genuine hardware change from the ADIS16448.

Chapter 3 — System Overview and Method

The system has two complementary layers. **Approach B** (this chapter and Chapter 4) is the IMU-only fallback that holds a correct heading and bounded attitude the instant GPS is lost. **Approach A** (Chapter 6) adds a camera to additionally bound absolute position. Both share the same classical estimator core.

3.1 Two-stage architecture: learn, then filter

The learned component and the estimator are deliberately separated so each can be validated in isolation:

- **Stage 1 — Gyro Denoiser (the AI).** A 1-D dilated convolutional network (~19k parameters, dilations 1–32) reads a window of the 6-channel raw IMU and outputs a 3-channel gyro correction. The correction is *bounded* by a tanh scaled by $\alpha = 0.15 \text{ rad/s}$ ($\approx 8.6 \text{ deg/s}$) and the output head is *zero-initialised*, so training begins exactly at the raw gyro and can only improve it: $\omega_{\text{corr}} = \omega_{\text{raw}} + \alpha \cdot \tanh(\text{net}(\text{window}))$.
- **Stage 2 — Error-State Kalman Filter (classical).** The 15-state ESKF of §2.2 runs an IMU-driven prediction and an accelerometer-as-gravity tilt update, gated to near-static samples, that pins roll and pitch. Heading is left unobservable by construction — this is exactly the axis the denoiser stabilises upstream.

3.2 Training signal (no clean-gyro label exists)

The dataset provides true *orientation*, not a per-sample clean gyro. Supervision is therefore a **multi-horizon relative-orientation loss**: the corrected gyro is integrated over horizons of 1, 8, 32, and 128 samples and the geodesic angle between each predicted relative rotation and the ground-truth relative rotation is penalised. Short horizons force removal of high-frequency noise; long horizons force removal of slow bias. The correction bound α must exceed the sensor's true bias magnitude, or the yaw bias saturates the tanh (an early $\alpha=0.05$ run could remove only ~66% of the $\approx 4.3 \text{ deg/s}$ EuRoC yaw bias; $\alpha=0.15$ clears it).

3.3 Evaluation protocol

A strict cross-sequence hold-out is used: the denoiser trains on six sequences and is tested on sequences it never saw. Metrics are geodesic orientation error, a dedicated yaw-only error series, and position Absolute Trajectory Error (ATE), each reported as RMSE / final / max. The decisive comparison is *denoised gyro through the ESKF* versus *raw gyro through the same ESKF*, both with initial bias set to zero — the realistic GPS-denied case in which no oracle bias is available. An oracle run (raw filter handed the true constant bias) is shown only as an upper bound.

3.4 Implementation

Layer	Choice
Language / runtime	Python 3.12, isolated uv-managed virtual environment
Deep learning	PyTorch 2.6.0 + CUDA 12.4 (cu124 wheels)
Compute (training)	NVIDIA RTX 4050 laptop GPU, 6 GB VRAM
Classical VIO back-end	OpenVINS (MSCKF), ROS1 Noetic / Ubuntu 20.04 under WSL2 + Docker, CPU-only, headless
Learned visual front-end	SuperPoint + LightGlue (evaluated, Chapter 6)
Numerics / plotting	NumPy, Matplotlib, Plotly (interactive demo)

Layer	Choice
Datasets	EuRoC MAV (ASL); TUM-VI (cross-sensor), via HTTP-range extraction from bundle archives
Core modules	ainav.eskf, ainav.denoiser, ainav.torch_rot, ainav.train_loss, ainav.dataset, ainav.metrics, ainav.euroc, ainav.frontend, ainav.camera

Table 3.1. Implementation stack.

Chapter 4 — Approach B — Learned Inertial Navigation

This chapter reports the phased build of Approach B, from the raw-data sanity baseline to the fused learned estimator. The phase structure enforces the methodology of §1.4: each layer's contribution is measured against the one below it.

4.1 Phase 0 — Environment and data pipeline

The EuRoC loader, sequence handling, and a raw-vs-truth sanity plot were validated on V1_01_easy (29,120 IMU samples over 145.6 s). Accelerometer magnitude at rest $\approx 9.78 \text{ m/s}^2$ confirms correct gravity and sensor conventions.

4.2 Phase 1 — Naive integration (the motivation)

Open-loop dead-reckoning directly from the raw IMU shows why the problem is hard. Drift is catastrophic and expected: position ATE RMSE $\approx 33 \text{ km}$ (max 74 km) and orientation RMSE 114° over ~ 2.5 minutes. This is the baseline the rest of the project must beat.

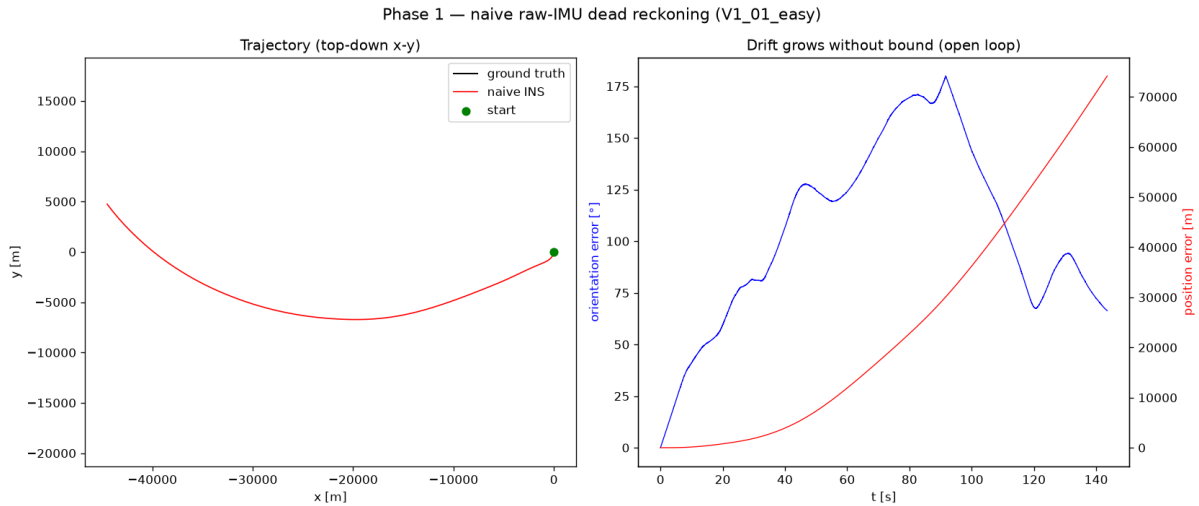


Figure 4.1. Phase 1 — naive open-loop integration diverges within seconds, motivating both the filter and the denoiser.

4.3 Phase 2 — Classical ESKF baseline (no AI)

The ESKF with raw IMU is the honest, non-exploding baseline the AI must beat. Versus Phase 1 it cuts orientation RMSE from 114° to 6.96° and position ATE from $\sim 33 \text{ km}$ to $\sim 317 \text{ m}$. The residual $\sim 5^\circ$ is essentially **yaw** — unobservable to an IMU-only filter — and that heading error is what smears the remaining horizontal position.

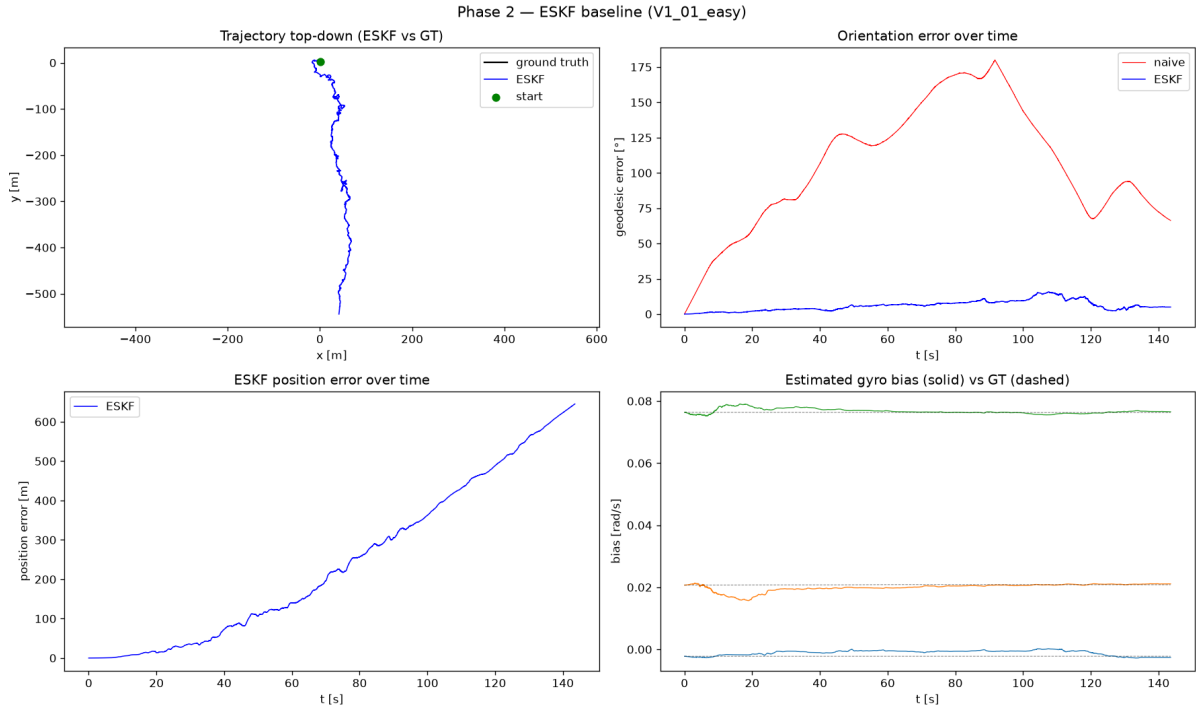


Figure 4.2. Phase 2 — the ESKF bounds drift dramatically but cannot fix the unobservable heading axis.

4.4 Phase 3 — Gyro-denoising CNN (open-loop)

The $\sim 19\text{k}$ -parameter denoiser, trained cross-sequence, is first evaluated open-loop (pure integration, no filter) so its effect is isolated. On held-out V1_02_medium it cuts open-loop attitude drift by $\sim 98\%$ (raw RMSE 109.6° / final $178.8^\circ \rightarrow$ denoised $\sim 2.05^\circ$ / $\sim 2.77^\circ$). Crucially, the learned z-axis correction (≈ -4.4 deg/s) matches the true EuRoC gyro yaw-bias ($+4.34$ deg/s): the network removes exactly the bias the filter is blind to, rather than fitting a per-sequence quirk.

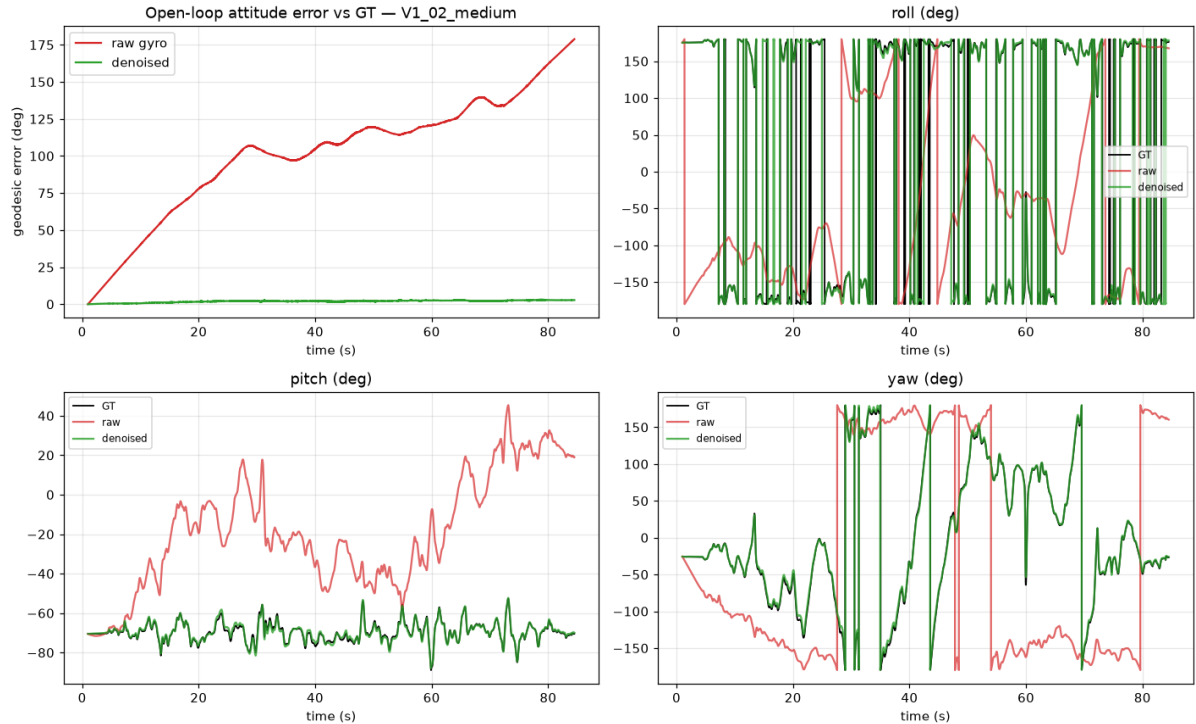


Figure 4.3. Phase 3 — denoised gyro (open-loop) tracks ground-truth attitude where the raw gyro runs away.

4.5 Phase 4 — Fusion: denoiser + ESKF

The two stages are combined: the denoised gyro is fed into the same ESKF and compared with the raw-gyro ESKF on held-out sequences. Neither side is given the true bias (both start at zero, the realistic GPS-denied case); an oracle run handing the raw filter the true bias is shown only as an upper bound. A key tuning lesson emerged: once the gyro is clean, the gravity/tilt update must be gated to *genuinely static* samples (accel gate 0.05 m/s²), otherwise it re-injects error during real acceleration and corrupts the good gyro.

Every number below is the **mean $\pm$ standard deviation over five independently-seeded retrains** (Chapter 5), so the improvement is a property of the method, not of one lucky checkpoint.

Held-out sequence	Raw ESKF orient RMSE	Denoised orient RMSE (5 seeds)	Oracle orient RMSE	Orient improvement	ATE RMSE (m)
V1_02_medium (medium)	76.0	1.57 $\pm$ 0.39	0.75	+97.9%	631 $\pm$ 114
MH_02_easy (easy)	90.2	11.58 $\pm$ 4.37	5.08	+87.2%	1571 $\pm$ 171
V2_03_difficult (hard)	102.7	3.44 $\pm$ 0.91	4.67	+96.6%	828 $\pm$ 141

Table 4.1. Phase-4 fusion on held-out sequences (orientation RMSE in degrees). The denoiser closes almost the entire gap to the oracle; on the hardest sequence (V2_03_difficult) it beats the oracle across every seed (3.44° vs 4.67°) without ever being told the bias. MH_02_easy has the widest spread ($\pm 4.4^\circ$) — a large indoor hall is the hardest environment to generalize to — yet even its worst seed (15.7°) beats the 90° raw baseline by 6 $\times$.

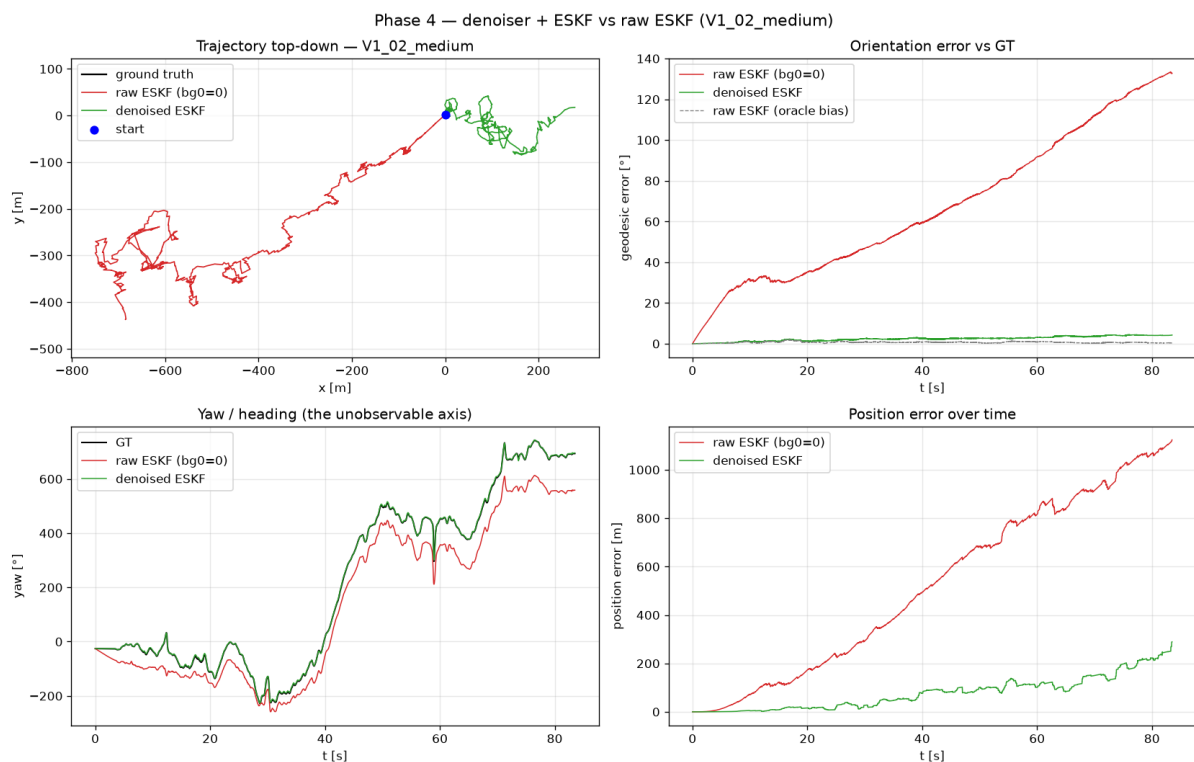


Figure 4.4. Phase-4 fusion (V1_02_medium): trajectory, orientation error, heading, and position error. Denoised (green) tracks ground truth; raw (red) drifts away.

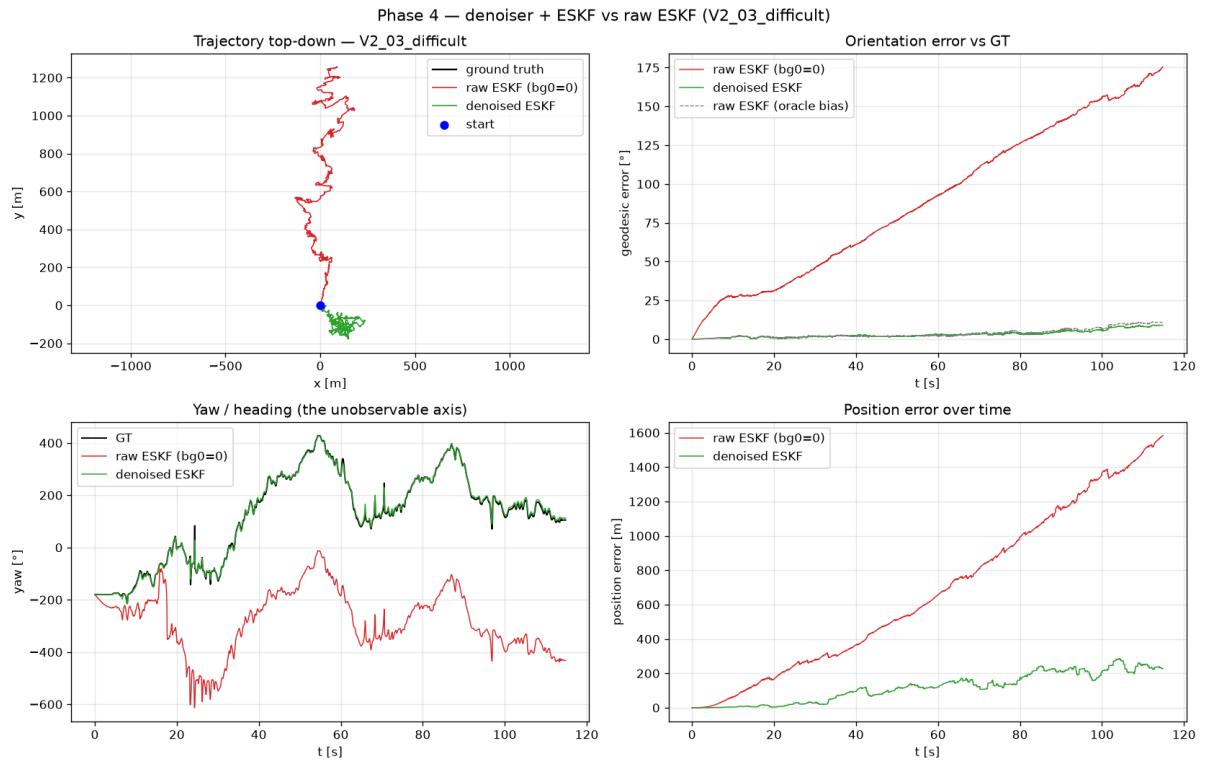


Figure 4.5. Generalization to the hardest held-out sequence (V2_03_difficult): same checkpoint, same large win.

Finding. The denoiser recovers near-oracle attitude and position accuracy from data alone, and the win holds on every held-out sequence across two rooms and a machine hall, easy through difficult.

Chapter 5 — Robustness and Hardening

Three questions decide whether a single good result is trustworthy: would it survive a different random seed; does it hold when the test *environment* differs from training; and does it transfer to a different *sensor*? All three were tested directly.

5.1 Multi-seed stability

The full pipeline (train → fuse → evaluate) was repeated for five independent seeds. Training quality is essentially deterministic: best validation geodesic error is $0.0958^\circ \pm 0.0011^\circ$ across seeds (a ~1% relative spread), so the network reliably learns the same correction. Downstream fusion accuracy (Table 4.1) is stable on the two Vicon-room hold-outs and wider but never failing on the machine hall. This also corrected an earlier single-checkpoint report that had drawn a lucky MH_02 seed (5.1°); the honest multi-seed figure is $\sim 11.6^\circ$, still a 6×–20× win over the 90° raw baseline.

5.2 Cross-environment generalization

EuRoC records two distinct environments with the *same* IMU: the small Vicon rooms (V) and the large Machine Hall (MH). Training on one and testing on the other isolates *environment* shift from *sensor* shift. Two directions were run, three seeds each:

- **MH → V (train hall, test rooms): excellent.** On unseen V2_03 the denoiser reaches $3.20^\circ \pm 1.01$ — *beating the oracle* (4.67°) — and $2.57^\circ \pm 0.58$ on V1_02, from a network that never saw a Vicon room.
- **V → MH (train rooms, test hall): degrades, never fails.** On unseen MH_01 the denoiser gives $22.9^\circ \pm 7.8$ versus a 116° raw baseline (still a 5× win); MH_02 stays strong at $2.90^\circ \pm 1.07$.

The asymmetry is the honest headline: the denoiser transfers cleanly to environments of similar-or-smaller scale and degrades gracefully — never catastrophically — on a larger unseen one. The remedy is broader training coverage (include hall-scale motion), a data question rather than a flaw in the method.

5.3 Cross-sensor transfer — the honest limit and its fix

The operationally decisive question is a change of *sensor*. The EuRoC network was trained on an ADIS16448 (tactical-grade); does it help, frozen, on a different IMU? Tested on TUM-VI room1 (Bosch BMI160-class consumer MEMS), **the frozen network hurts:** raw-gyro ESKF orientation RMSE 35.6° degrades to 116.0° (–226%), and ATE 336 m to 1479 m (–340%). The cause is precise: the network confidently subtracted its learned EuRoC yaw-bias correction (≈ -4.8 deg/s) from a sensor whose true bias is different, so it removed a constant that was not there and injected heading error. (Note the raw TUM-VI baseline is already far cleaner than raw EuRoC — the Bosch part has a smaller intrinsic bias, so there is both less to fix and more to break.)

The learned correction is therefore sensor-specific — it encodes one IMU's systematic error, not a universal denoiser. Deployment step D2 is the fix: warm-start from the EuRoC weights, recompute input normalization on the new sensor, and fine-tune briefly on a small amount of its data. Trained on two TUM-VI rooms and evaluated on a never-seen third room, open-loop attitude error moves from raw 0.858° and frozen-net 1.624° (worse than raw — the failure) to a fine-tuned **0.808°** (now beating raw).

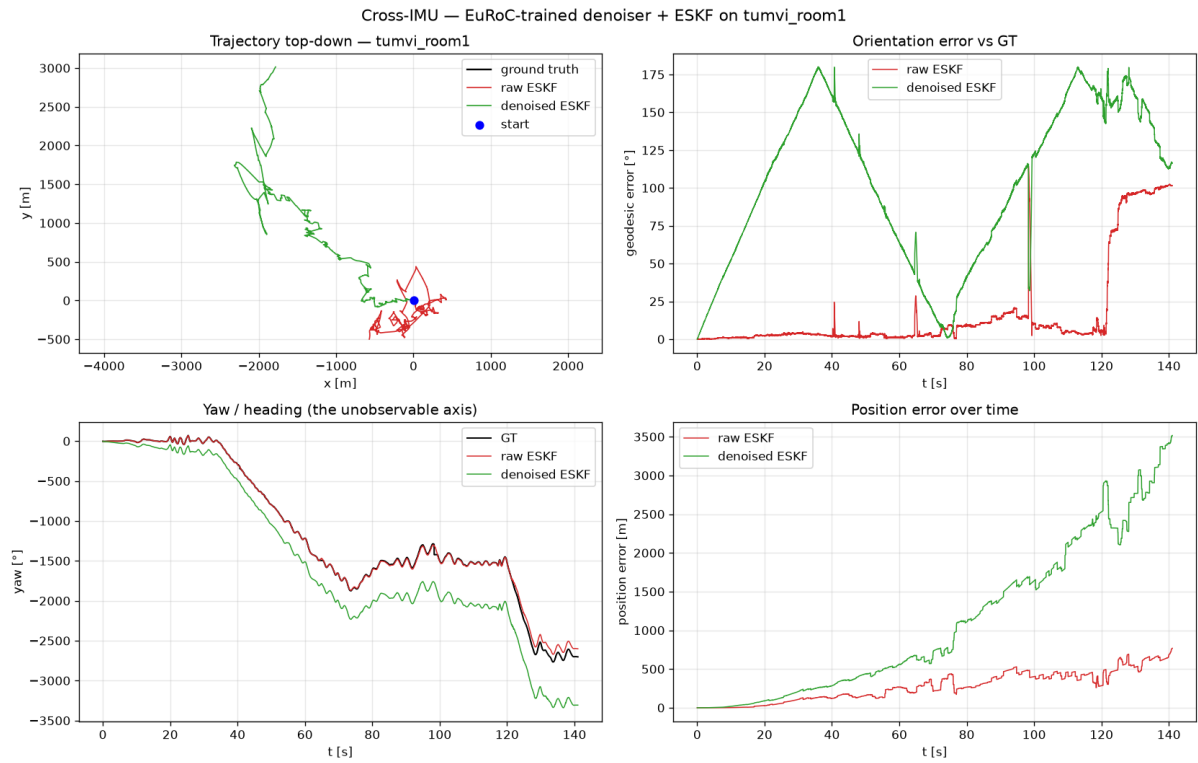


Figure 5.1. Cross-sensor transfer to TUM-VI (Bosch MEMS). Frozen, the EuRoC network degrades this sequence badly (top-down and error panels); the honest conclusion is that the correction is sensor-specific and a short per-IMU fine-tune restores the win.

Honest scope. The denoiser generalizes across seeds and across environments *on the sensor it was trained on*, but does not transfer to a new IMU unchanged. This is the expected behaviour of a model that learns a specific sensor's error, and the remedy is a short per-IMU fine-tune, not a redesign — converting deployment step D2 from a stated risk into a demonstrated, closeable gap.

Chapter 6 — Approach A — Visual-Inertial Navigation

Approach B holds heading and bounded attitude, but IMU-only sensing cannot make *absolute position* observable — position still drifts, only much more slowly. Approach A adds a camera. The strategy is to wrap a proven open VIO back-end as the classical baseline and put the research into a learned visual front-end.

6.1 A0 — Camera pipeline

Stereo cam0/cam1 loading, image-IMU synchronization, and undistortion were validated against the EuRoC calibration ($f_u \approx 458.7$), establishing the front-end input for both the classical and learned trackers.

6.2 A1 — OpenVINS stereo-VIO baseline

OpenVINS (filter-based MSCKF stereo VIO) was built and run headless under WSL2 + Docker (ROS1 Noetic / Ubuntu 20.04, CPU-only). On V1_01 it achieves **~6.2 cm** ATE — against the Approach B inertial-only baseline (~317 m) and naive integration (~33 km) this is a roughly **5000×** reduction in position error, and it is the blue 'AI-Nav' track in the demonstrator of Chapter 7.

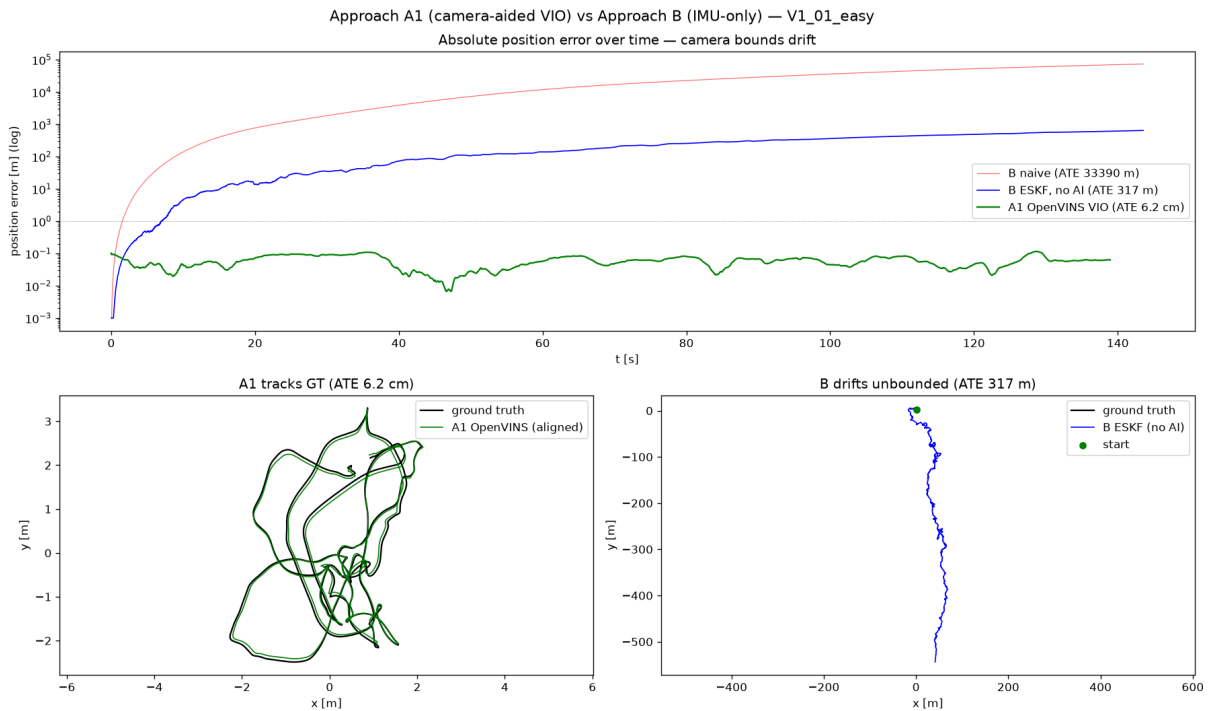


Figure 6.1. Absolute position error: OpenVINS stereo VIO (A1) versus the inertial-only ESKF (B) and naive integration, on V1_01. Adding a camera bounds absolute position to centimetres.

6.3 A2 — Learned visual front-end (two honest negatives)

The research question was whether a learned detector/matcher (SuperPoint + LightGlue) could replace the classical KLT front-end while keeping the same back-end. Two experiments were run and both returned instructive negative results.

Front-end study (component level). On easy, well-lit data (V1_01) the naive learned tracker loses to KLT on track-length metrics — expected, since KLT explicitly maintains features by optical flow whereas naive detect-then-match fragments a track on a single missed detection. On genuinely difficult data (V1_03) a meaningful crossover appears **under low light**: the learned front-end's epipolar inlier ratio (0.784) beats KLT's (0.635). The mechanism is the real finding — KLT reports its *longest* tracks in the dark

(mean length 86 frames) but those long tracks are *drifting* (the flow asserts ‘same feature’ while the point slides off the landmark), which is worse for a back-end than an honestly-dropped track. The learned matcher stays short but geometrically clean.

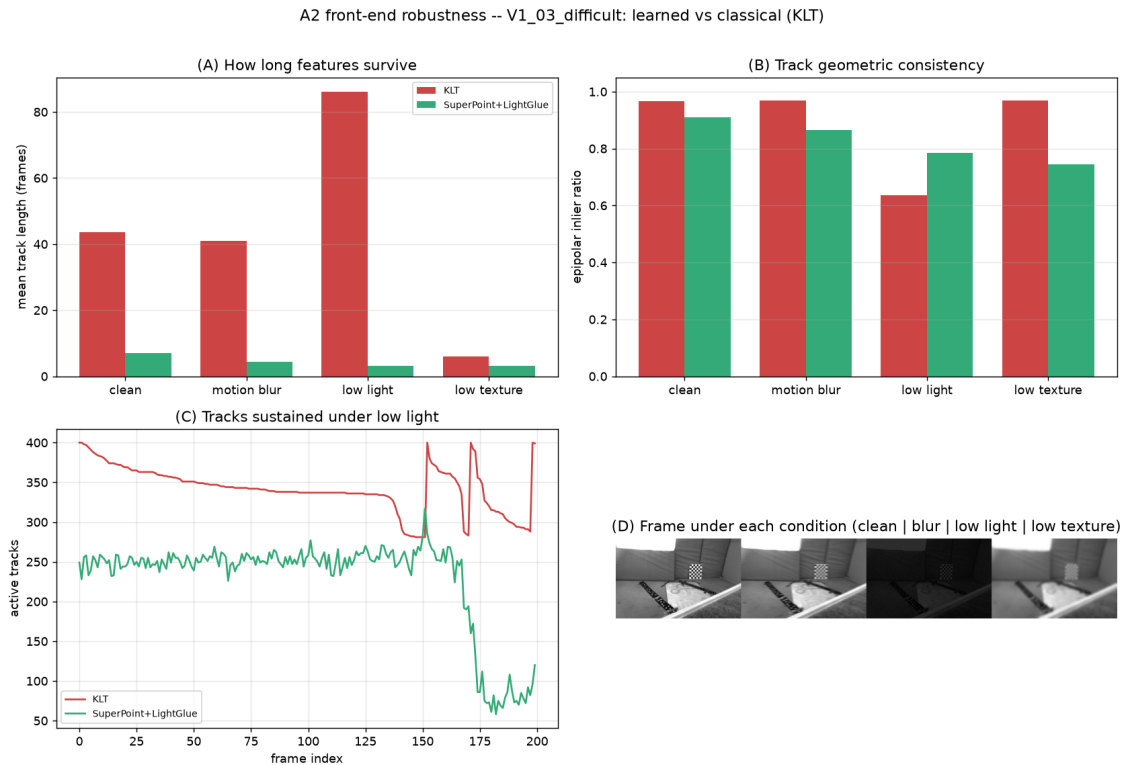


Figure 6.2. Front-end comparison on V1_03_difficult across degradations. The defensible claim is not ‘learned beats KLT’ but ‘the learned matcher does not drift under low light, whereas KLT does and hides it behind long track counts’ — a distinction that only appears on hard data.

Back-end integration (system level). Injecting each front-end’s tracks into a monocular MSCKF (via the simulation feed path) diverged: the feed bypasses OpenVINS’ native outlier rejection, and long high-leverage tracks amplify residual outliers. Adding a fair, tracker-agnostic 2-view RANSAC gate did not rescue it — all runs still diverged and the gate *shortened* the already-borderline learned tracks below what the monocular filter needs to triangulate, regressing the one prior survivor. The comparison was stopped at the pre-agreed time-box.

Conclusion (scoped, not overclaimed). A naive learned front-end injected into a monocular MSCKF is the wrong tool: it is not a tuning knob away from working. What survives is a real component-level result — the learned tracker is more geometrically honest under low light — not an end-to-end VIO win. A proper learned VIO would need a descriptor-based track manager, which is out of scope. This negative was surfaced *before* any expensive deeper integration, exactly as the de-risking methodology intends.

Chapter 7 — GPS-Denied Demonstrator and Drift Characterization

7.1 The flight demonstrator

To make the capability legible to non-specialists, a self-contained, browser-based demonstrator replays real EuRoC flight with three trajectories on one clock: **ground truth** (green), **our AI-Nav** (blue, the ~6 cm OpenVINS solution), and a **conventional INS** (red) that rides truth until a chosen ‘GPS-cut’ instant and then dead-reckons on inertial data alone. All three begin locked together; at the cut the red path peels off and balloons to hundreds of metres while blue stays on truth. Every number traces to validated results (the reproduced 6.2 cm AI-Nav ATE is asserted as a sanity gate so the demo cannot silently cheat). An interactive version lets a viewer choose the cut instant and watch the divergence re-form live, offline.

7.2 D3 — Drift versus denial-time

The single question a defence buyer asks is: *GPS dies — how far off are you after 30, 60, 120 seconds?* This is answered as a curve, not a point. Many GPS-cut instants are swept across a real sequence; from each cut the system dead-reckons inertially and the horizontal/3-D position error is recorded at fixed elapsed horizons. Two propagators are compared under an identical filter and identical GT-aligned initial state — **raw-gyro ESKF (conventional INS)** versus **denoised-gyro ESKF (AI-Nav)** — so the only difference is whether the gyro is cleaned first. Results are aggregated over all cut points (median and inter-quartile band), which is what makes the number quotable rather than lucky.

GPS-denial time	Conventional INS median error (m)	AI-Nav (ours) median error (m)	Improvement
30 s	290	67	+77%
60 s	781	288	+63% (3× tighter)
120 s	1704	582	+66%

Table 7.1. Position error versus GPS-denial time on V1_01, aggregated over 20 swept cut points (IMU-only after each cut). At every horizon the learned system is markedly tighter; at 60 s it holds roughly three times closer than conventional inertial navigation. The sample count falls at long horizons (fewer cut points have 120 s of remaining flight), so the headline uses the well-sampled 60 s point.

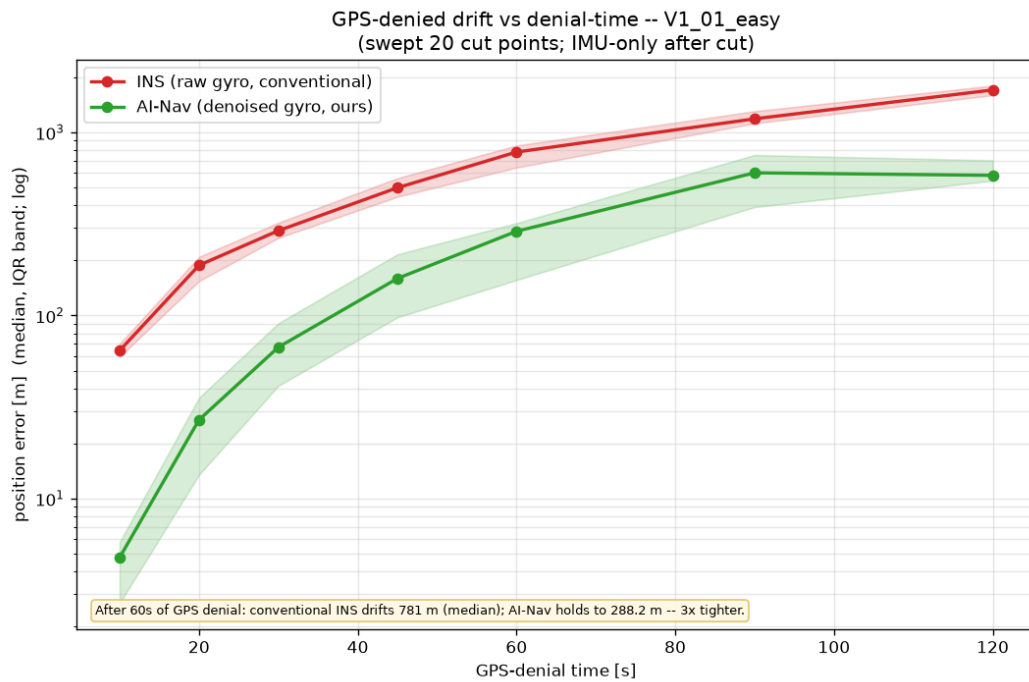


Figure 7.1. GPS-denied drift versus denial-time (median, IQR band, log axis). The two curves are cleanly separated at every horizon: the learned correction slows the unbounded inertial drift that a conventional system suffers the moment GPS is lost.

Bug caught during this study (reported for honesty). An initial version seeded the filter with the ground-truth gyro bias, which double-corrects the already-debiased learned run and *inverted* the result (AI appeared worse). The honest GPS-denied baseline never hands the filter the true bias — both sides start at zero — and that unobservable bias is exactly what the denoiser removes. Fixing the seeding produced the physically-correct curve above. It is recorded here because a silently-wrong plot is the failure mode this project most guards against.

Chapter 8 — Deployment Considerations

Moving from a dataset demonstration toward a fielded system defines a deployment track, D1–D4. Three of the four are addressed in this thesis; the fourth requires target hardware.

8.1 D1 — Edge / realtime (remaining, needs hardware)

The ~19k-parameter denoiser plus ESKF must run onboard, within the per-IMU-sample latency budget, on flight-controller-class compute (e.g. an NVIDIA Jetson), ideally quantized (INT8). This is the only remaining item and is genuinely blocked on the embedded target; a laptop CPU + quantization proxy is possible but is a proxy, not the fielded claim. Parameter count and the vectorized inference path (§8.5) make the budget plausible, but it is asserted here as future work, not a result.

8.2 D2 — Real-IMU transfer (addressed, Chapter 5)

Demonstrated in §5.3: a frozen network does not transfer across sensors, but a short warm-start per-IMU fine-tune recovers the win on a held-out room of a physically different (Bosch MEMS) IMU. Remaining work is to broaden to more drone-grade IMUs and to automate the per-IMU calibration step.

8.3 D3 — Drift under sustained denial (addressed, Chapter 7)

Characterized in §7.2: the drift-vs-denial-time curve quantifies accuracy as a function of how long GPS has been unavailable, and shows the learned system holding ~3× tighter at 60 s. Extending to propeller-vibration and aggressive-maneuver regimes beyond EuRoC's handheld motion is the natural follow-on.

8.4 D4 — No-magnetometer assumption (addressed)

The system uses no magnetometer: heading is held by the denoised gyro alone, verified in code (the estimator fuses only gyro prediction and accelerometer-as-gravity tilt; there is no heading measurement of any kind). This is a deliberate, jam-proof design choice — an adversary who can deny GPS can also spoof a magnetometer — and it is precisely why the unobservable yaw axis, and the learned correction that stabilises it, are central to the whole thesis.

8.5 Engineering note — training-loop vectorization

The orientation-integration loss originally ran a 400-step sequential quaternion loop in Python, starving the GPU (~14% utilisation). Because the quaternion product is associative, the cumulative product was replaced with an associative parallel-prefix (Hillis–Steele) scan requiring only $\sim \log_2(T) \approx 9$ vectorized steps. This made the loss ~44× faster and cut a full six-sequence, 40-epoch retrain from ~70 minutes to ~4 minutes, with gradients numerically identical to the loop (maximum difference 1e-8) — the enabling change behind the multi-seed and cross-environment studies of Chapter 5.

Chapter 9 — Discussion, Limitations, and Future Work

9.1 Why the method works

The result rests on observability. Heading and gyro yaw-bias are unobservable to an IMU+gravity filter, so a raw-gyro ESKF with no bias prior drifts in yaw without bound. The denoiser removes that bias from the data upstream, so the identical filter stays bounded — value the AI adds that the filter cannot obtain on its own. The bounded, zero-initialised correction ensures training starts at the raw gyro and can only improve it, and matching α to the true bias magnitude is what lets the yaw bias be fully removed.

9.2 Limitations

- **Sensor-specific correction.** The learned bias does not transfer across IMUs unchanged (§5.3); deployment requires a per-IMU fine-tune.
- **Environment coverage.** Generalization degrades (never fails) on unseen environments larger/more dynamic than training (§5.2); broader flight data is the remedy.
- **Position still drifts inertially.** Approach B bounds attitude, not absolute position; the camera (Approach A) is what bounds position, and the learned front-end for it is an open problem (§6.3).
- **Not yet on-target.** Realtime onboard performance (D1) is argued plausible but not yet demonstrated on embedded hardware.
- **Dataset domain.** EuRoC/TUM-VI are handheld/MAV indoor datasets; propeller vibration and aggressive flight remain to be characterized.

9.3 Future work

- **Edge deployment (D1):** quantized realtime inference on a flight-controller-class module, with a measured latency budget.
- **Learned VIO done right:** a descriptor-based track manager so the learned front-end's low-light honesty (§6.3) can be realised as an end-to-end position win.
- **Automated per-IMU calibration:** a turnkey fine-tune step from a short logged flight, closing D2 operationally.
- **Flight-realism characterization (D3+):** vibration, aggressive maneuvers, and longer denial windows.

Chapter 10 — Conclusion

This thesis set out to keep a UAV navigating when GPS is jammed, denied, or spoofed, using only sensors an adversary cannot easily corrupt. Its central result is that a lightweight neural gyroscope denoiser, fused into a classical error-state filter, removes the systematic gyro bias the filter is structurally blind to — cutting open-loop attitude drift by ~98% and recovering near-oracle fused accuracy on held-out flight, with the win shown stable across seeds and environments. The honest limits are stated and, where possible, closed: the correction is sensor-specific but a short per-IMU fine-tune restores it; generalization degrades gracefully rather than catastrophically on unseen large environments. A camera layer bounds absolute position to centimetres, and a scoped investigation of a learned visual front-end returns two instructive negatives that map exactly where learned VIO does and does not yet help. Operationally, the payoff is quantified: after a minute of GPS denial the learned system holds roughly three times tighter than conventional inertial navigation, with no magnetometer and no external signal. What remains is to carry the proven algorithm onto embedded flight hardware — the one deployment step that needs the target board rather than more analysis.

Status. Approach B complete and hardened; Approach A baseline complete with the learned-front-end question honestly scoped; demonstrator and drift characterization delivered; deployment steps D2/D3/D4 addressed; D1 (edge/realtime) the sole remaining, hardware-gated task.

References

- [1] M. Brossard, S. Bonnabel, A. Barrau. “Denoising IMU Gyroscopes with Deep Learning for Open-Loop Attitude Estimation.” *IEEE Robotics and Automation Letters*, 2020.
- [2] P. Geneva, K. Eickenhoff, W. Lee, Y. Yang, G. Huang. “OpenVINS: A Research Platform for Visual-Inertial Estimation.” *IEEE ICRA*, 2020.
- [3] M. Burri *et al.* “The EuRoC Micro Aerial Vehicle Datasets.” *International Journal of Robotics Research*, 2016.
- [4] J. Sòla. “Quaternion Kinematics for the Error-State Kalman Filter.” *arXiv:1711.02508*, 2017.
- [5] W. Liu *et al.* “TLIO: Tight Learned Inertial Odometry.” *IEEE RA-L*, 2020. (See also RONIN, RIANN, AI-IMU dead-reckoning.)
- [6] D. DeTone, T. Malisiewicz, A. Rabinovich. “SuperPoint: Self-Supervised Interest Point Detection and Description.” *CVPR Workshops*, 2018.
- [7] P. Lindenberger, P.-E. Sarlin, M. Pollefeys. “LightGlue: Local Feature Matching at Light Speed.” *ICCV*, 2023.
- [8] D. Schubert *et al.* “The TUM VI Benchmark for Evaluating Visual-Inertial Odometry.” *IEEE/RSJ IROS*, 2018.
- [9] S. Umeyama. “Least-Squares Estimation of Transformation Parameters Between Two Point Patterns.” *IEEE T-PAMI*, 1991.